자료구조

08 주차

트리



8장. 트리



8.1 트리란?

8.2 이진트리

8.3 이진트리의 연산

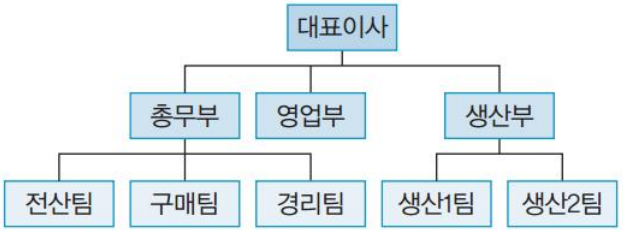
8.4 이진트리 응용: 모스 코드 결정트리

8.5 힙 트리

8.6 힙의 응용: 허프만 코드

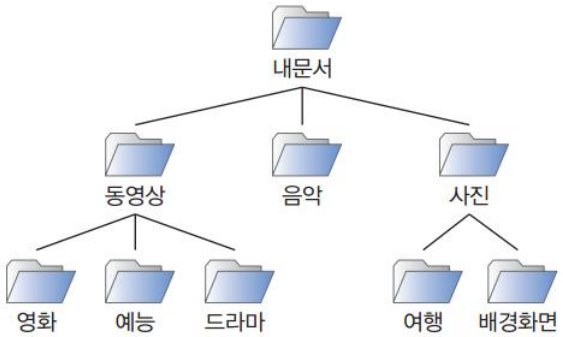
8.1 트리란?

- 계층적인 자료의 표현에 적합한 자료 구조

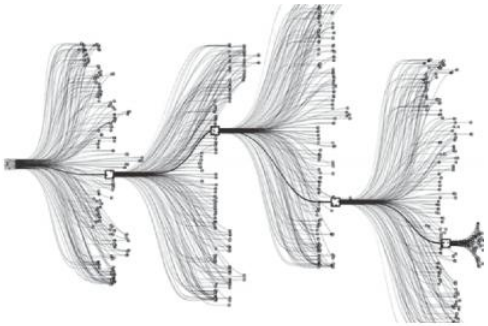


(a) 회사의 조직도



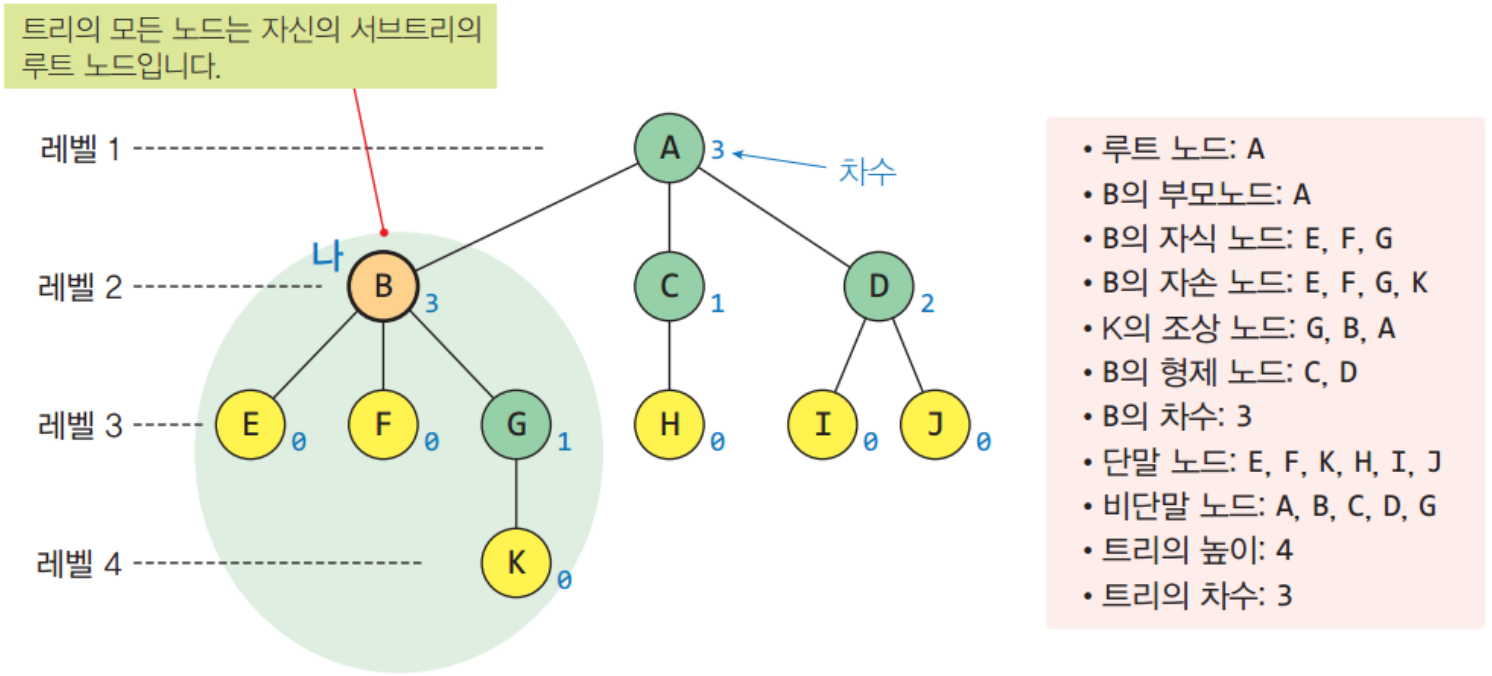


(b) 컴퓨터의 폴더 구조



(c) 인공지능 바둑 프로그램의
거대한 결정 트리(decision tree)

트리의 용어



- 루트 노드: A
- B의 부모노드: A
- B의 자식 노드: E, F, G
- B의 자손 노드: E, F, G, K
- K의 조상 노드: G, B, A
- B의 형제 노드: C, D
- B의 차수: 3
- 단말 노드: E, F, K, H, I, J
- 비단말 노드: A, B, C, D, G
- 트리의 높이: 4
- 트리의 차수: 3

- 루트 노드
- 간선 또는 엣지
- 부모 / 자식 / 형제
- 조상 / 자손
- 단말 / 비단말 노드

- 노드의 차수
- 트리의 차수
- 레벨
- 트리의 높이
- 포레스트(forest)

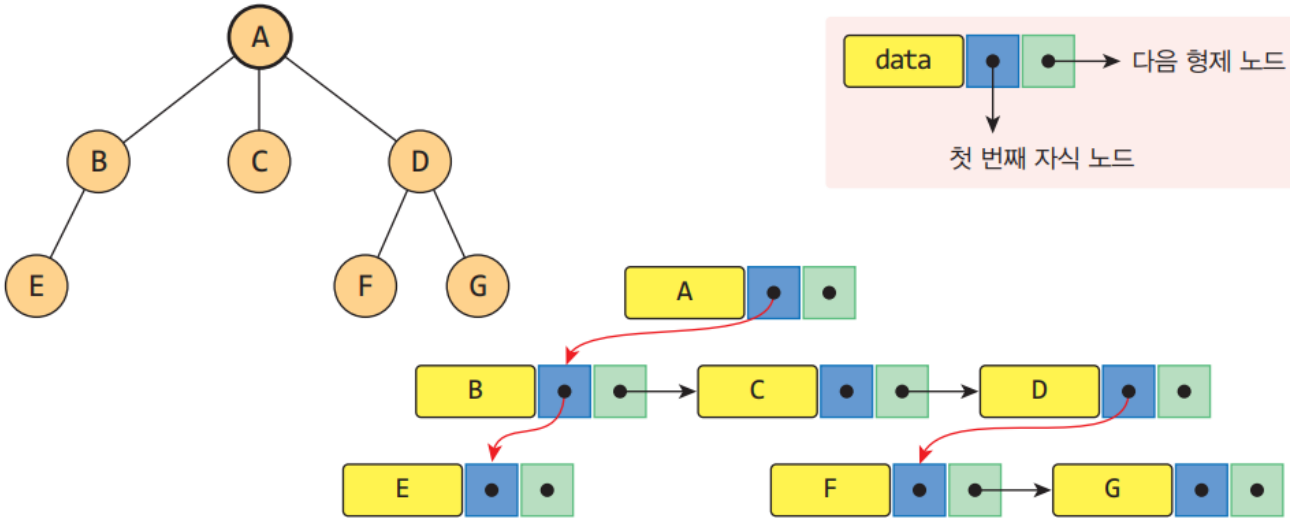
일반 트리의 표현 방법



- 일반 트리의 노드

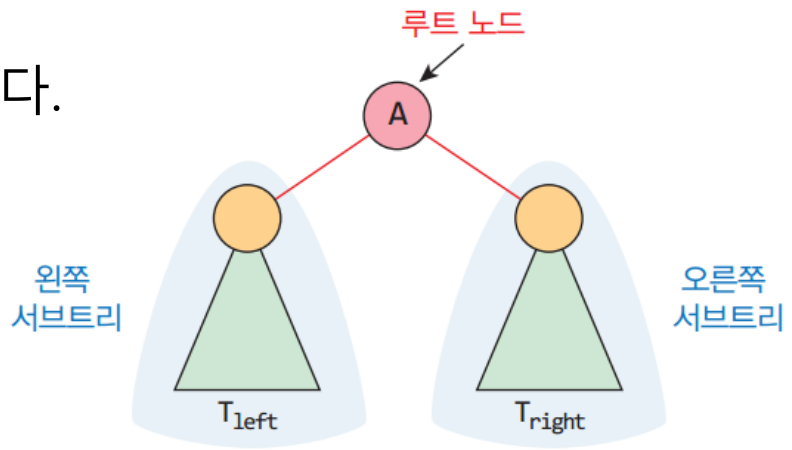


- 다른 방법: child-sibling



8.2 이진 트리

- 모든 노드가 2개의 서브 트리를 갖는 트리
  - 서브 트리는 공집합일 수 있다.
  - 이진 트리는 순환적으로 정의된다.



정의 8.1 이진 트리의 정의

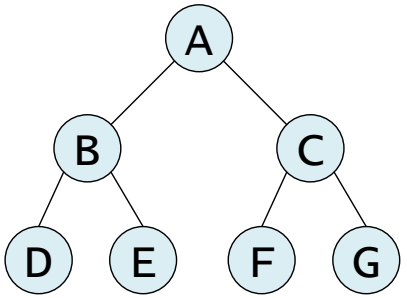
(1) 공집합이거나

(2) 루트와 왼쪽 서브 트리, 오른쪽 서브 트리로 구성된 노드들의 집합. 이진트리의 서브 트리들은 모두 이진트리이어야 함.

검색 → BST | 우선순위 → Heap | 압축 → Huffman | 수식 처리 → Expression Tree

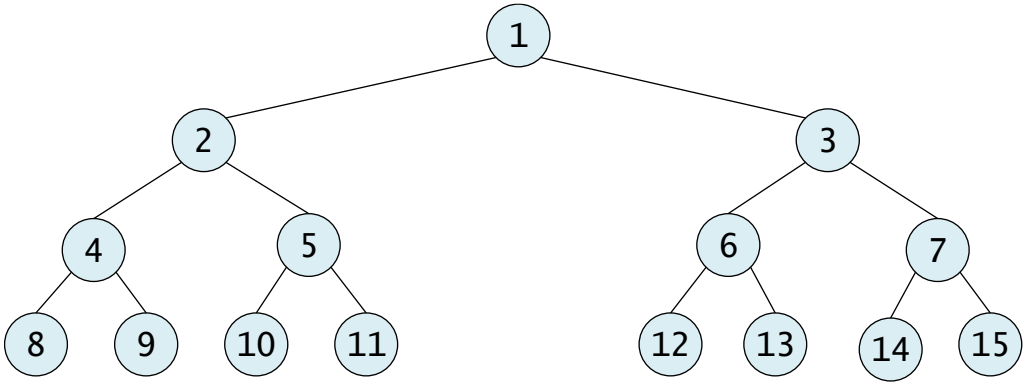
이진 트리의 종류

- 포화 이진 트리(full binary tree)
  - 트리의 각 레벨에 노드가 꽉 차있는 이진 트리

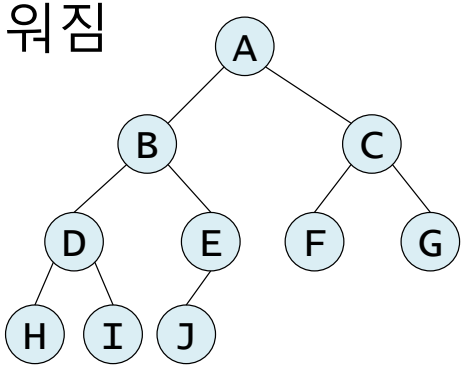


전체 노드 개수 : $2^{1-1}+2^{2-1}+2^{3-1}+\cdots+2^{k-1}=\sum_{i=0}^{k-1}2^i=2^k-1$

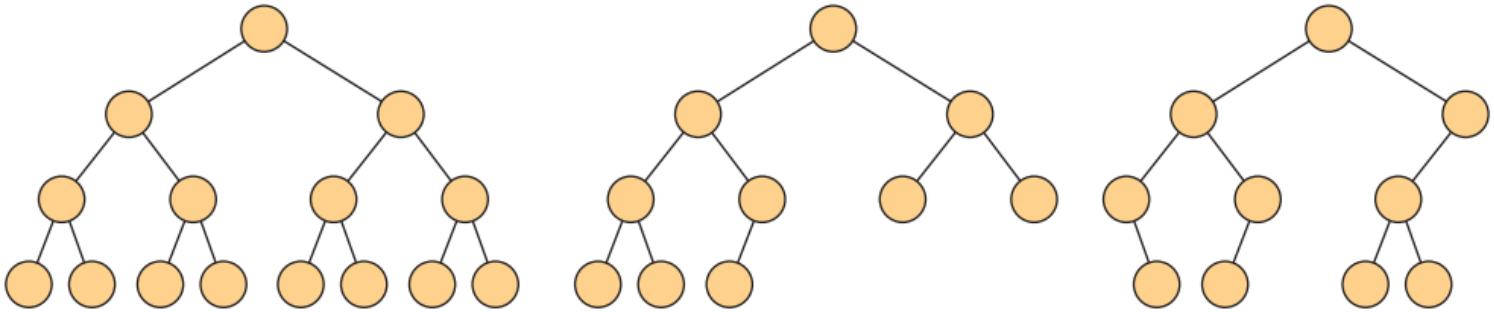
  - 노드의 번호



- 완전 이진 트리(complete binary tree)
  - 높이가 h일 때 레벨 1부터 h-1까지는 노드가 모두 채워짐
  - 마지막 레벨 h에서는 노드가 순서대로 채워짐
  - 예: 힙 트리



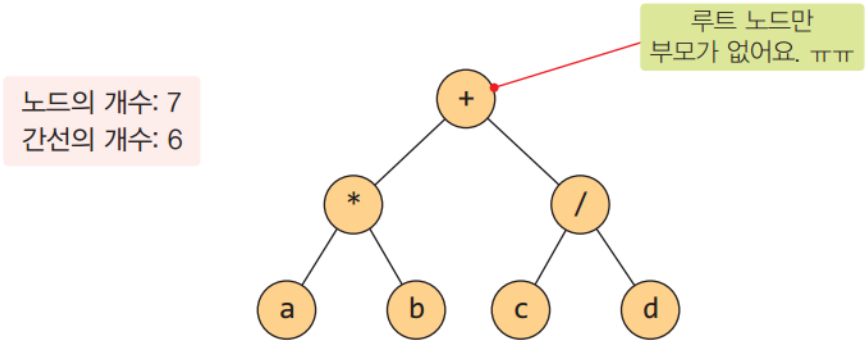
- 이진 트리 예



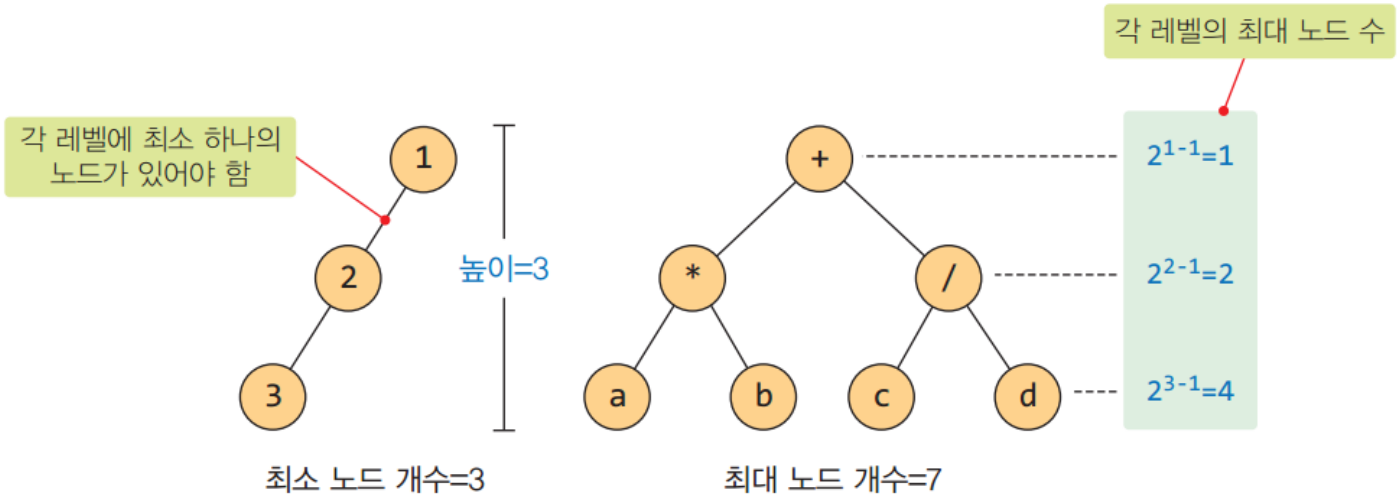
이진 트리의 성질



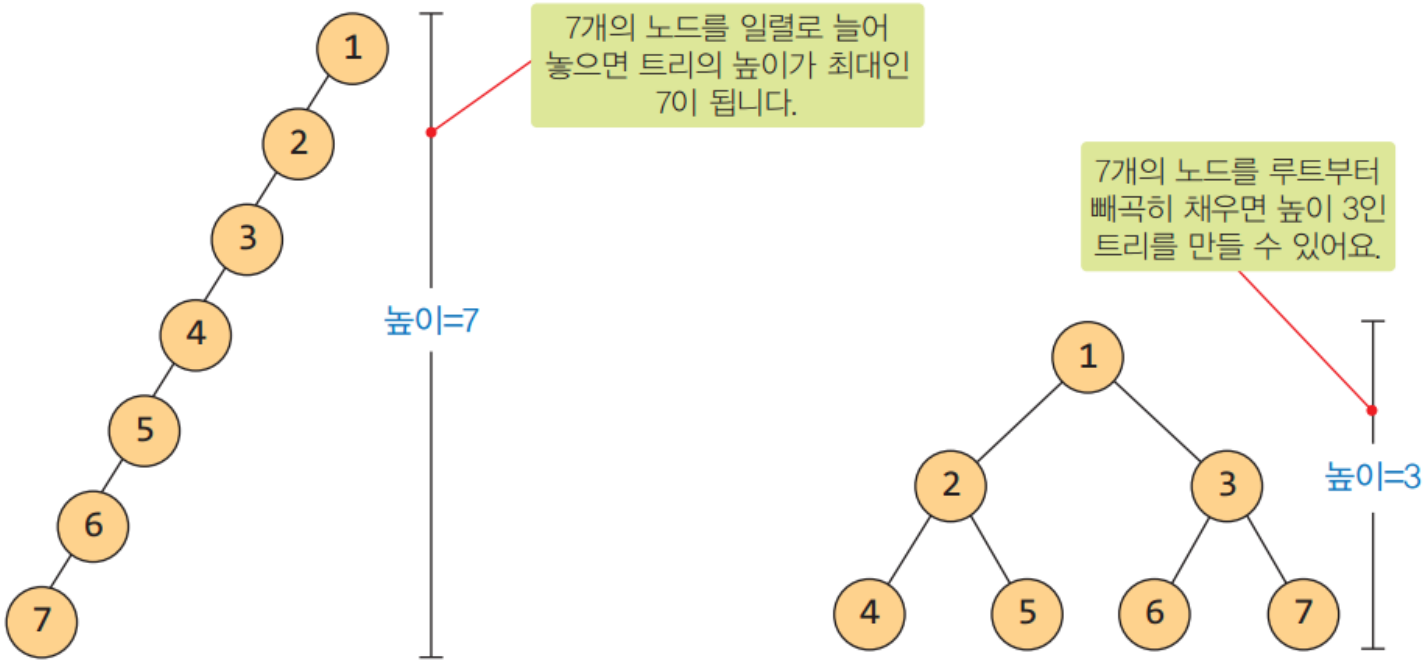
- 노드의 개수가 n개이면 간선의 개수는 n-1



- 높이가 h 이면 $h \sim 2^h - 1$개의 노드를 가짐

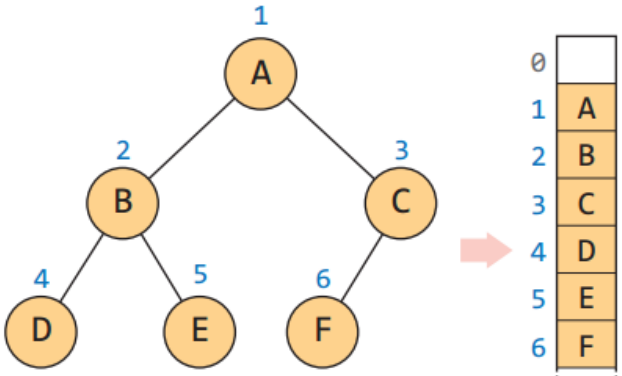


- n개 노드의 이진 트리 높이: $\lceil log_2(n+1) \rceil \sim n$

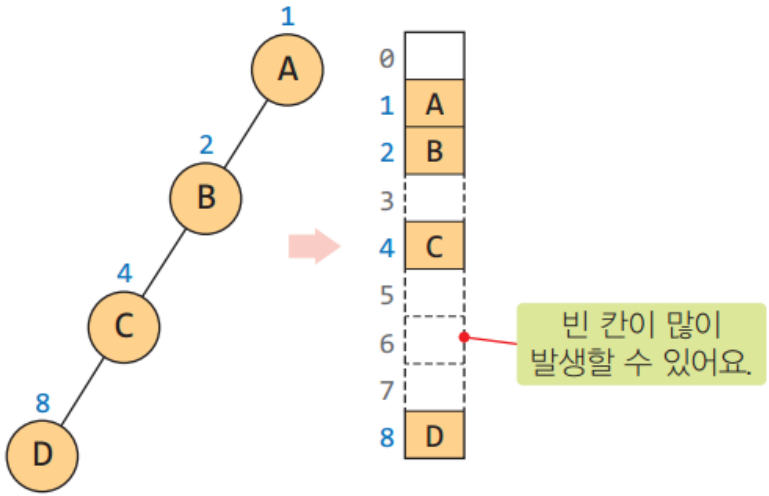


이진트리의 표현 방법

- 배열 표현법



완전이진트리의 배열 표현

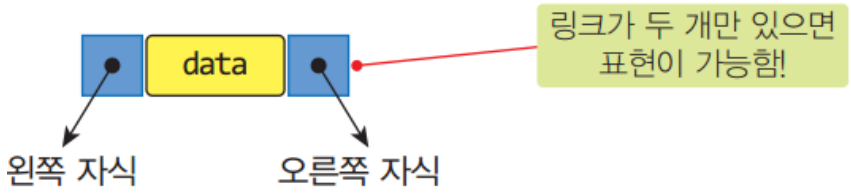


경사이진트리의 배열 표현

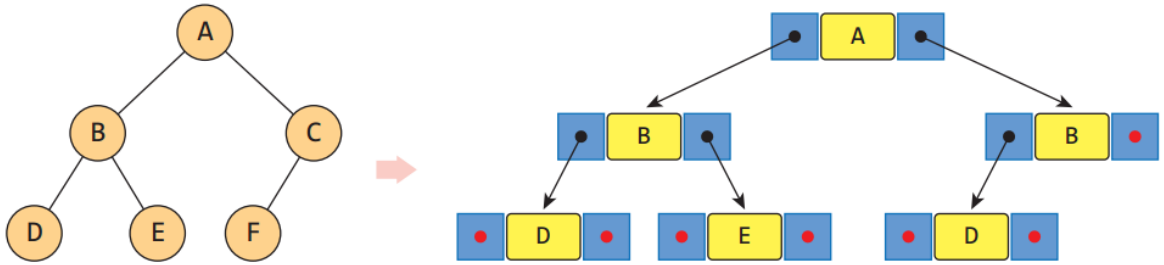
- 노드 i의 부모 노드 인덱스 = i/2
- 노드 i의 왼쪽 자식 노드 인덱스 = 2i
- 노드 i의 오른쪽 자식 노드 인덱스 = 2i+1

파이썬에서는 나눗셈 연산자가
/와 //로 구분되어 있습니다.
정수 나눗셈을 위해서는
i//2를 써야 합니다.

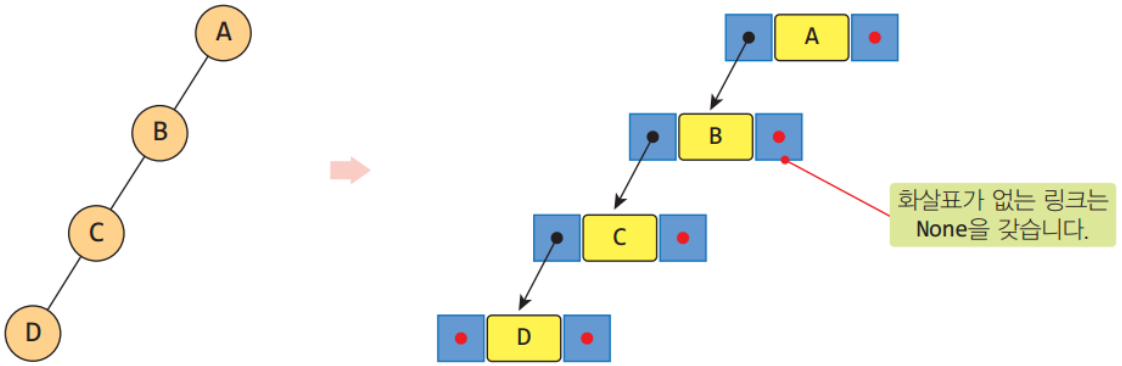
- 링크 표현법




```
class TNode:
    def __init__ (self, data, left, right):
        self.data = data
        self.left = left
        self.right = right
```



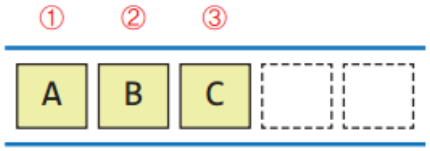
완전이진트리의 링크 표현



경사이진트리의 링크 표현

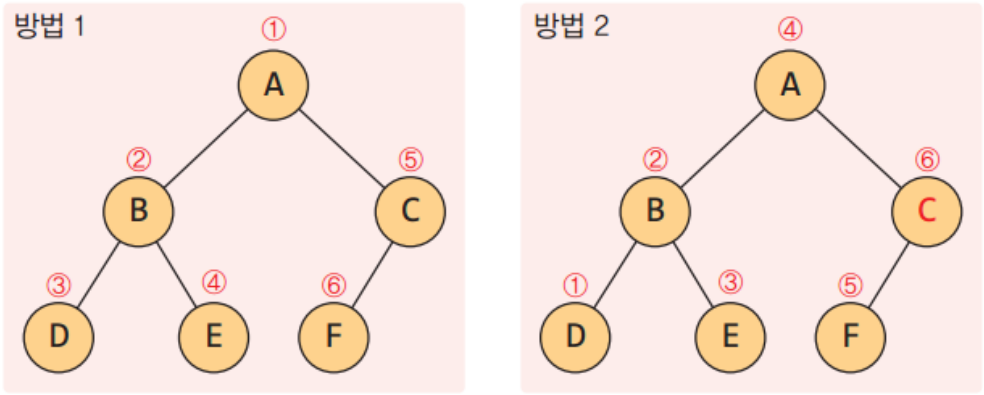
8.3 이진트리의 연산

- 순회(traversal)
  - 트리의 모든 노드를 한 번씩 방문하는 작업
  - 선형 자료구조는 순회가 단순



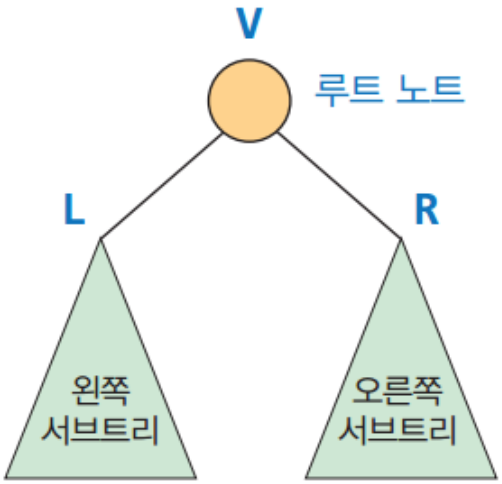
선형자료구조는
순회 방법이 단순하다.

  - 트리는 다양한 방법이 있음



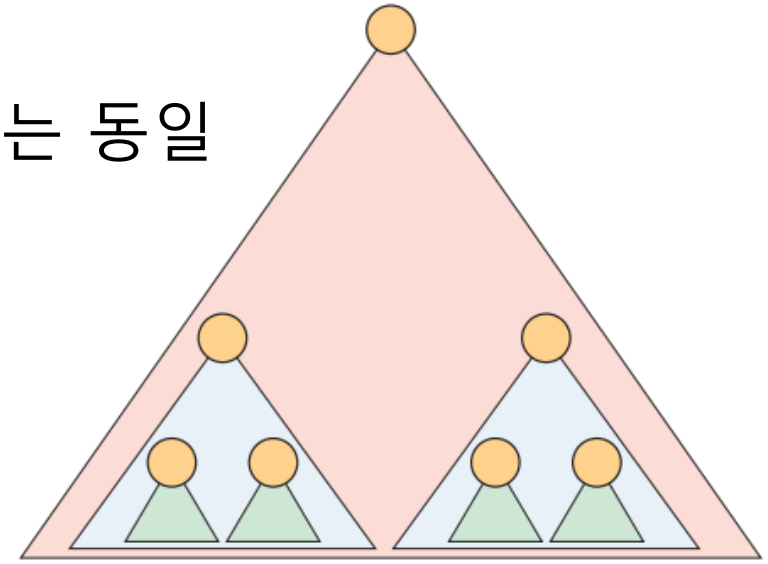
트리는 다양한 방법으로 순회할 수 있다.

이진트리의 기본 순회 (순환 구조)

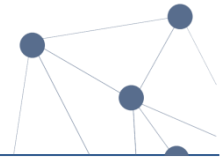


- 전위 순회(preorder traversal) : VLR
- 중위 순회(inorder traversal) : LVR
- 후위 순회(postorder traversal) : LRV

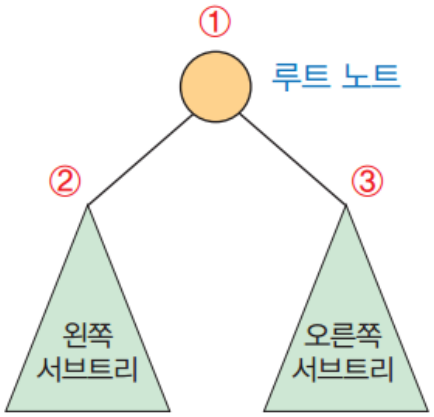
- 전체 트리나 서브 트리나 구조는 동일



전위 순회



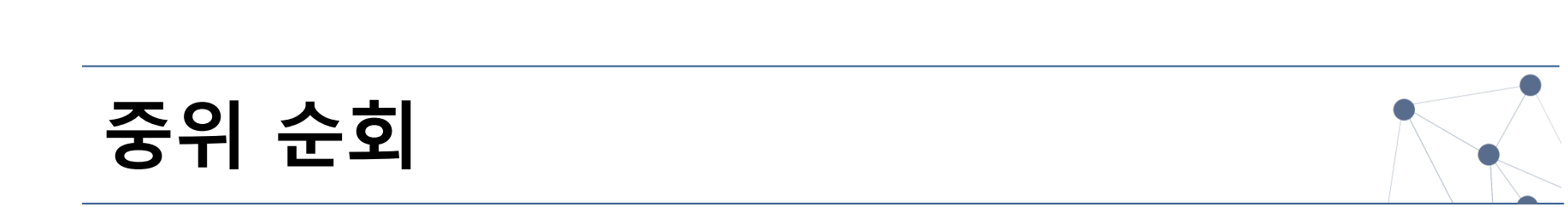
- 루트 → 왼쪽 서브트리 → 오른쪽 서브트리



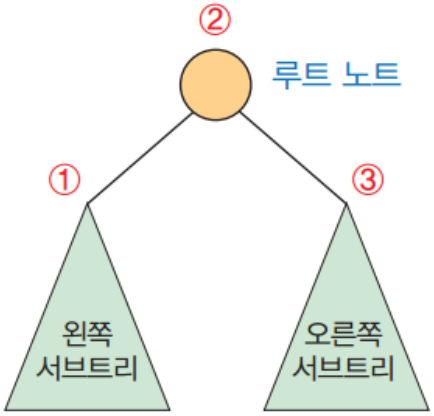

```
def preorder(n) :                    # 전위 순회 함수
    if n is not None :
        print(n.data, end=' ')       # 먼저 루트노드 처리(화면 출력)
        preorder(n.left)             # 왼쪽 서브트리 처리
        preorder(n.right)            # 오른쪽 서브트리 처리
```

- 응용 예
  - 트리 구조 복사/표현
  - 노드의 레벨 계산
  - 구조화된 문서 출력

중위 순회



- 왼쪽 서브트리 → 루트 → 오른쪽 서브트리



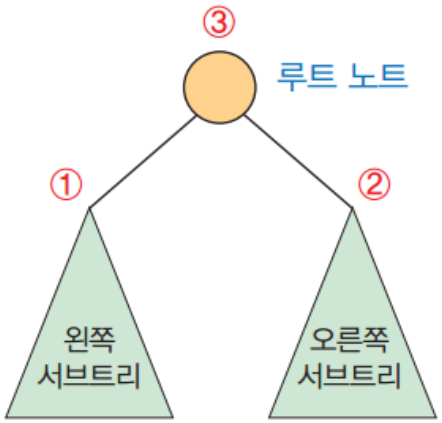

```
def inorder(n) :                    # 전위 순회 함수
    if n is not None :
        inorder(n.left)             # 왼쪽 서브트리 처리
        print(n.data, end=' ')      # 루트노드 처리(화면 출력)
        inorder(n.right)            # 오른쪽 서브트리 처리
```

- 응용 예
  - 정렬 (이진 탐색 트리)

후위 순회



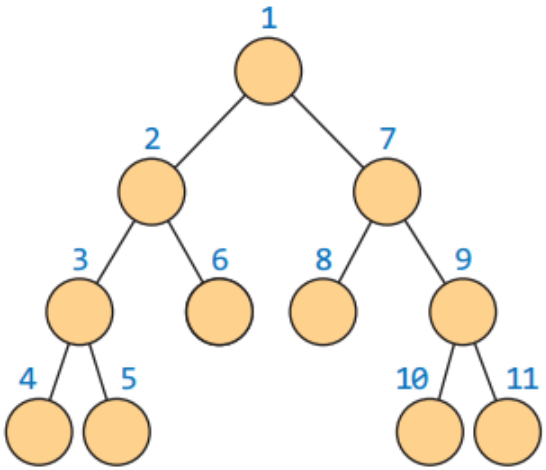
- 왼쪽 서브트리 → 오른쪽 서브트리 → 루트



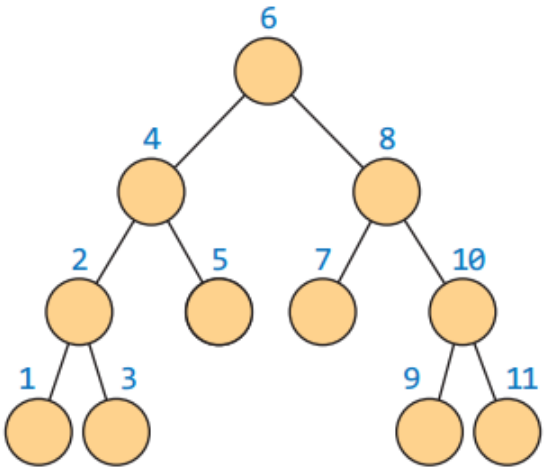

```
def postorder(n) :
    if n is not None :
        postorder(n.left)
        postorder(n.right)
        print(n.data, end=' ')
```

- 응용 예
  - 트리 삭제
  - 폴더 용량 계산

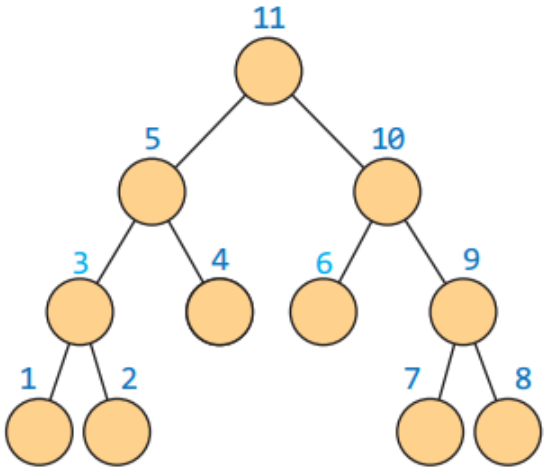
순회 방법에 따른 노드 방문 순서



(a) 전위 순회



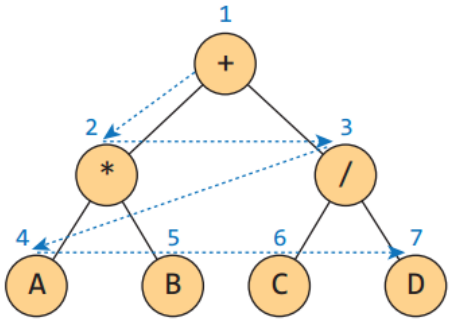
(b) 중위 순회

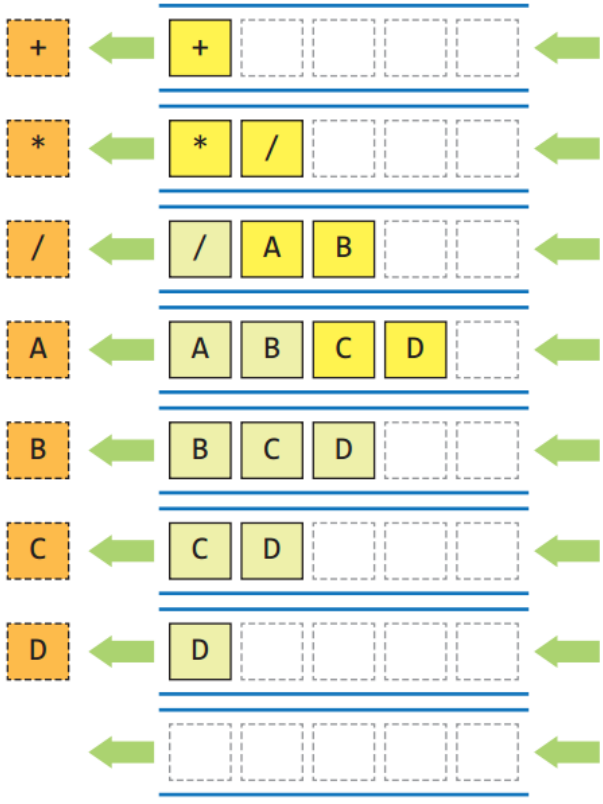


(c) 후위 순회

레벨 순회

- 노드를 레벨 순으로 검사하는 순회방법
  - 큐를 사용해 구현
  - 순환을 사용하지 않음





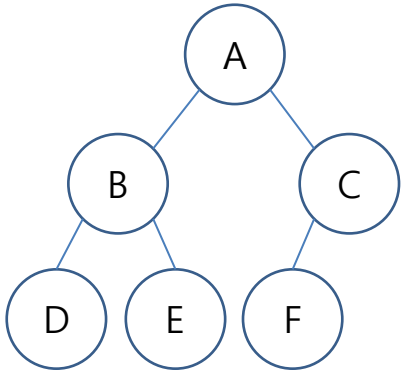
레벨 순회 사전 작업



클래스 생성자

```
class TNode:          # 이진 트리를 위한 노드 클래스
    def __init__ (self, elem, left, right):
        self.data = elem
        self.left = left
        self.right = right

    def isLeaf(self):    # 왼쪽/오른쪽 없으면 리프 노드
        return self.left is None and self.right is None
```



main 부분

```
if __name__ == "__main__":

    d = TNode('D', None, None)
    e = TNode('E', None, None)
    b = TNode('B', d, e)
    f = TNode('F', None, None)
    c = TNode('C', f, None)
    root = TNode('A', b, c)
```

d 와 e가 각각 노드 클래스('B')의 left, right

f 혼자 노드 클래스('C')의 left

b 와 c가 각각 노드 클래스('A')의 left, right

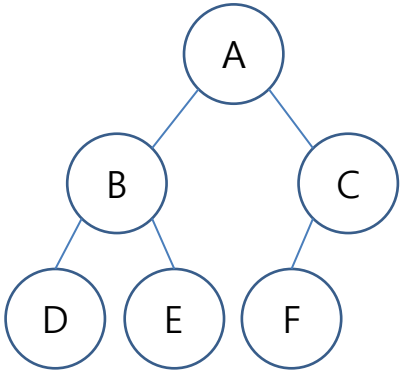
레벨 순회 알고리즘



레벨 순회

```
from CircularQueue import CircularQueue

def levelorder(root) :
    queue = CircularQueue()
    queue.enqueue(root)
    while not queue.isEmpty() :
        n = queue.dequeue()
        if n is not None :
            print(n.data, end=' ')
            queue.enqueue(n.left)
            queue.enqueue(n.right)
```



큐 객체 초기화
최초의 큐에는 루트 노드만.
큐가 공백 상태 될 때 까지 맨 앞에서 노드 n 꺼냄

노드 정보 출력
왼쪽 자식 노드 큐에 삽입
오른쪽 자식 노드 큐에 삽입

-**while 문** 진행-

A(root) 꺼내와서(dequeue) print 하고 -> B, C enqueue
-> 큐 안에 있는 B 먼저 꺼내와서 print 하고 D, E enqueue
-> 큐 안에 있는 C 꺼내와서 print 하고 F enqueue
나머지 D, E, F는 left/right 가 없으니 본인 print 만 되고 끝

queue.enqueue(root)부분
[A] 상태로 시작:

A <-빠짐 [B, C]
B <-빠짐 [C, D, E]
C <-빠짐 [D, E, F]
D, E, F 차례대로 빠짐

전체 노드 수, 단말 노드의 수 구하기



- 전체 노드 수

```
def count_node(n) :      # 순환을 통해 트리의 노드 수 구하는 함수
    if n is None :       # n이 None 이면 공백 트리 -> 0 반환
        return 0
    else :               # 좌우 서브 트리 노드 수의 합 +1(루트) 반환
        return 1 + count_node(n.left) + count_node(n.right)
```

- 단말 노드의 수

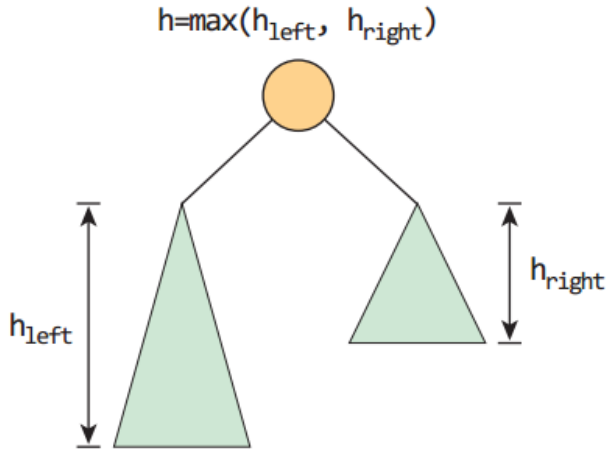
```
def count_leaf(n) :
    if n is None : return 0          # 공백트리 : 0 반환
    elif n.isLeaf() : return 1       # 단말노드 : 1 반환
    else : return count_leaf(n.left) + count_leaf(n.right)
                    # 비 단말노드 : 좌우 서브트리 결과의 합 반환
```

트리 높이 구하기



- 트리의 높이

```
def calc_height(n) :
    if n is None : return 0
    hLeft = calc_height(n.left)      # 왼쪽 트리 높이
    hRight = calc_height(n.right)    # 오른쪽 트리 높이
    if (hLeft > hRight) : return hLeft + 1
    else: return hRight + 1          # 둘 중 큰거 +1(루트)
```



테스트 코드

아래 코드 전체 복붙

```
# 코드 8.1: 이진트리를 위한 노드 클래스
class TNode:          # 이진 트리를 위한 노드 클래스
    def __init__ (self, elem, left, right):
        self.data = elem
        self.left = left
        self.right = right

    def isLeaf(self):   # 왼쪽/오른쪽 없으면 리프 노드
        return self.left is None and self.right is None

# 코드 8.2: 이진트리의 전위순회
def preorder(n) :
    if n is not None :
        print(n.data, end=' ')
        preorder(n.left)
        preorder(n.right)

# 코드 8.3: 이진트리의 중위순회
def inorder(n) :
    if n is not None :
        inorder(n.left)
        print(n.data, end=' ')
        inorder(n.right)

# 코드 8.4: 이진트리의 후위순회
def postorder(n) :
    if n is not None :
        postorder(n.left)
        postorder(n.right)
        print(n.data, end=' ')

# 코드 8.5: 이진트리의 레벨순회
from CircularQueue import CircularQueue

def levelorder(root) :
    queue = CircularQueue()
    queue.enqueue(root)
    while not queue.isEmpty() :
        n = queue.dequeue()
        if n is not None :
            print(n.data, end=' ')
            queue.enqueue(n.left)
            queue.enqueue(n.right)

# 코드 8.6: 이진트리의 노드 수 계산
def count_node(n) :    # 순환을 통해 트리의 노드 수 구하는 함수
    if n is None :     # n이 None 이면 공백 트리 -> 0 반환
        return 0
    else :             # 좌우 서브 트리 노드 수의 합 +1(루트) 반환
        return 1 + count_node(n.left) + count_node(n.right)

# 코드 8.7: 이진트리의 단말노드 수 계산
def count_leaf(n) :
    if n is None : return 0          # 공백트리 : 0 반환
    elif n.isLeaf() : return 1       # 단말노드 : 1 반환
    else : return count_leaf(n.left) + count_leaf(n.right)
                # 비 단말노드 : 좌우 서브트리 결과의 합 반환

# 코드 8.8: 이진트리의 트리의 높이 계산
def calc_height(n) :
    if n is None : return 0
    hLeft = calc_height(n.left)     # 왼쪽 트리 높이
    hRight = calc_height(n.right)   # 오른쪽 트리 높이
    if (hLeft > hRight) : return hLeft + 1
    else: return hRight + 1         # 둘 중 큰거 +1(루트)


# 코드 8.9: 이진 트리 연산 테스트 프로그램
# __name__ == "__main__":

    print("₩n======= 이진트리 테스트 ====================================")
    d = TNode('D', None, None)
    e = TNode('E', None, None)
    b = TNode('B', d, e)
    f = TNode('F', None, None)
    c = TNode('C', f, None)
    root = TNode('A', b, c)

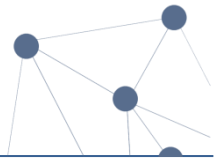
    print('₩n   In-Order : ', end='')
    inorder(root)
    print('₩n  Pre-Order : ', end='')
    preorder(root)
    print('₩n Post-Order : ', end='')
    postorder(root)
    print('₩nLevel-Order : ', end='')
    levelorder(root)
    print()

    print(" 노드의 개수 = %d개" % count_node(root))
    print(" 단말의 개수 = %d개" % count_leaf(root))
    print(" 트리의 높이 = %d" % calc_height(root))
```

```
======= 이진트리 테스트 ====================================

   In-Order : D B E A F C
  Pre-Order : A B D E C F
 Post-Order : D E B F C A
Level-Order : A B C D E F
 노드의 개수 = 6개
 단말의 개수 = 3개
 트리의 높이 = 3
```

8.4 응용: 모스 코드 결정트리



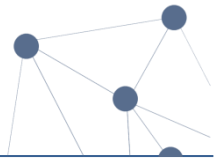
- 모스 부호
  - 1830년대 미국의 사뮤엘 모스(Samuel Morse)
  - 도트(점)와 대시(선)의 조합으로 구성된 메시지 전달용 부호

- 모스 부호 표

A .-	B -...	C -.-.	D -..	E .	F ..-.
G --.	H	I ..	J .---	K -.-	L .-..
M --	N -.	O ---	P .--.	Q --.-	R .-.
S ...	T -	U ..-	V ...-	W .--	X -..-
Y -.--	Z --..				

  - 예: SOS → ... --- ...

인코딩: 알파벳에서 모스 코드로 변환

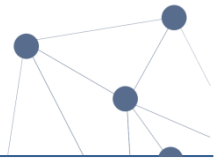


- 인코딩
  - 알파벳에서 모스 코드로 변환하는 작업
  - 표에서 코드의 위치를 바로 찾을 수 있음: $O(1)$
    - 예: 'C'의 위치: 'C' – 'A' = 2(위치)

```
table =[('A', '.-'),    ('B', '-...'),  ('C', '-.-.'),  ('D', '-..'),
        ('E', '.'),     ('F', '..-.'),  ('G', '--.'),   ('H', '....'),
        ('I', '..'),    ('J', '.---'),  ('K', '-.-'),   ('L', '.-..'),
        ('M', '--'),    ('N', '-.'),    ('O', '---'),   ('P', '.--.'),
        ('Q', '--.-'),  ('R', '.-.'),   ('S', '...'),   ('T', '-'),
        ('U', '..-'),   ('V', '...-'),  ('W', '.--'),   ('X', '-..-'),
        ('Y', '-.--'),  ('Z', '--..') ]

def encode(ch):
    idx = ord(ch)-ord('A')          # 리스트에서 해당 문자의 인덱스
    return table[idx][1]            # 해당 문자의 모스 부호 반환
```

디코딩: 모스 코드에서 알파벳 변환



- 디코딩
  - 모스 코드에서 해당 알파벳을 찾는 작업
  - 방법 1: 표에서 모든 코드를 순차 탐색: $O(n)$
    - 예) .--. -.-- - .... --- -. ???

```
def decode_slow(code):
    for e in table:
        if code == e[1] :
            return e[0]
    return None
```

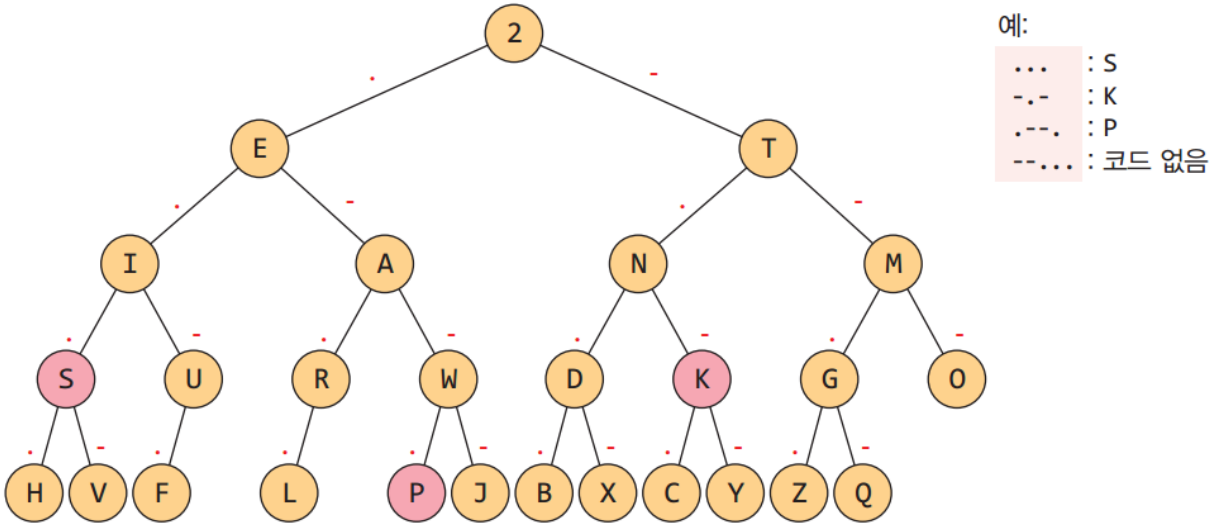
테이블의 모든 모스 코드(e[1])와 비교해, 찾는 코드가 있으면 그 코드의 알파벳(e[0]) 반환

  - 성능을 개선할 수 있는 방법? → 결정 트리

결정트리를 이용한 모스 코드의 디코딩

- 결정 트리(decision tree)란?
  - 여러 단계의 복잡한 조건을 갖는 문제에 대해 조건과 그에 따른 해결방법을 트리 형태로 나타낸 것

- 모스 코드를 위한 결정 트리



  - 디코딩 시간 복잡도: 트리의 높이에 비례 → $O(logn)$

모스 코드 결정트리 만들기



- 결정 트리 생성 알고리즘

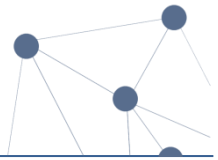
  ① 빈 루트 노드를 만든다. 한 번에 하나의 문자를 추가한다.

  ② 문자를 추가할 때 루트부터 시작하여 트리를 타고 내려간다. 만약 타고 내려갈 자식 노드가 None이면 새로운 노드를 추가한다. 이 노드의 문자는 아직 결정되지 않았다.

  ③ 마지막 코드의 노드에 도달하면 그 노드에 문자를 할당한다.

  – 코드 8.12

모스 코드 결정트리 만들기



아래 코드 전체 복붙

```
# 코드 8.10: 모스 코드 표와 인코딩 알고리즘
table =[('A', '.-'),    ('B', '-...'),  ('C', '-.-.'),  ('D', '-..'),
        ('E', '.'),     ('F', '..-.'),  ('G', '--.'),   ('H', '....'),
        ('I', '..'),    ('J', '.---'),  ('K', '-.-'),   ('L', '.-..'),
        ('M', '--'),    ('N', '-.'),    ('O', '---'),   ('P', '.--.'),
        ('Q', '--.-'),  ('R', '.-.'),   ('S', '...'),   ('T', '-'),
        ('U', '..-'),   ('V', '...-'),  ('W', '.--'),   ('X', '-..-'),
        ('Y', '-.--'),  ('Z', '--..') ]

class TNode:
    def __init__ (self, elem, left, right):
        self.data = elem
        self.left = left
        self.right = right

# 코드 8.12: 모스 코드 결정 트리 만들기
def make_morse_tree():
    root = TNode( None, None, None )
    for tp in table :
        code = tp[1]                    # 알파벳 말고 모스 코드
        node = root
        for c in code :                 # 맨 마지막 문자 이전까지 --> 이동
            if c == '.' :               # 왼쪽으로 이동
                if node.left == None :  # 비었으면 빈 노드 만들기
                    node.left = TNode (None, None, None)
                node = node.left        # 그쪽으로 이동
            elif c == '-' :             # 오른쪽으로 이동
                if node.right == None : # 비었으면 빈 노드 만들기
                    node.right = TNode (None, None, None)
                node = node.right       # 그쪽으로 이동

        node.data = tp[0]               # 코드의 알파벳
    return root

def encode(ch):
    idx = ord(ch)-ord('A')
    return table[idx][1]

def decode(root, code):
    node = root
    for c in code :
        if c == '.' :
            node = node.left
        elif c=='-' :
            node = node.right
    return node.data


# 코드 8.13: 모르스 코드 테스트 프로그램
if __name__ == "__main__":
    morseCodeTree = make_morse_tree()
    str = input("입력 문장(대문자) : ")
    mlist = []
    for ch in str:
        code = encode(ch)
        mlist.append(code)
    print("Morse Code: ", mlist)
    print("Decoding  : ", end='')
    for code in mlist:
        ch = decode(morseCodeTree, code)
        print(ch, end='')
    print()
```

```python
def make_morse_tree():
    root = TNode( None, None, None )
    for tp in table :
        code = tp[1]                    # 알파벳 말고 모스 코드
        node = root
        for c in code :                 # 모스코드 있는 동안 아래 반복
            if c == '.' :               # 왼쪽으로 이동
                if node.left == None :  # 비었으면 빈 노드 만들기
                    node.left = TNode (None, None, None)
                node = node.left        # 그쪽으로 이동
            elif c == '-' :             # 오른쪽으로 이동
                if node.right == None : # 비었으면 빈 노드 만들기
                    node.right = TNode (None, None, None)
                node = node.right       # 그쪽으로 이동

        node.data = tp[0]               # 코드의 알파벳
    return root
```

```python
def encode(ch):
    idx = ord(ch)-ord('A')
    return table[idx][1]
```

```python
def decode(root, code):
    node = root
    for c in code :
        if c == '.' :
            node = node.left
        elif c=='-' :
            node = node.right
    return node.data
```

< 입력 받은 문자에 해당하는 모스코드 리턴

< root :
morseCodeTree = make_morse_tree() 로 main에서 만들고, 해당 객체를 main에서 decode(morseCodeTree, code)로 호출

8.5 힙 트리



- 힙(Heap)이란?
  - "더미"와 모습이 비슷한 완전이진트리 기반의 자료 구조
  - 가장 큰(또는 작은) 값을 빠르게 찾아내도록 만들어진 자료 구조
  - 최대 힙, 최소 힙

- 힙의 정의

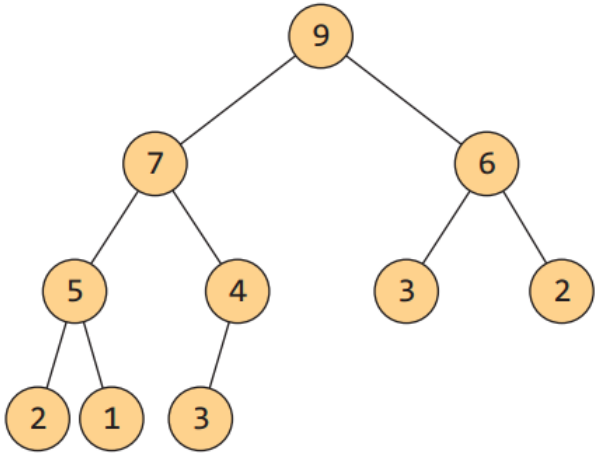
  - 최대 힙(max heap): 부모 노드의 키 값이 자식 노드의 키 값보다 크거나 같은 완전이진트리 (key(부모노드) $\geq$ key(자식노드))
  - 최소 힙(min heap): 부모 노드의 키 값이 자식 노드의 키 값보다 작거나 같은 완전이진트리 (key(부모노드) $\leq$ key(자식노드))

작업 스케줄링, 네트워크 패킷 우선순위, 운영체제의 우선순위 프로세스 등 **우선순위 큐** 적용 분야 사용

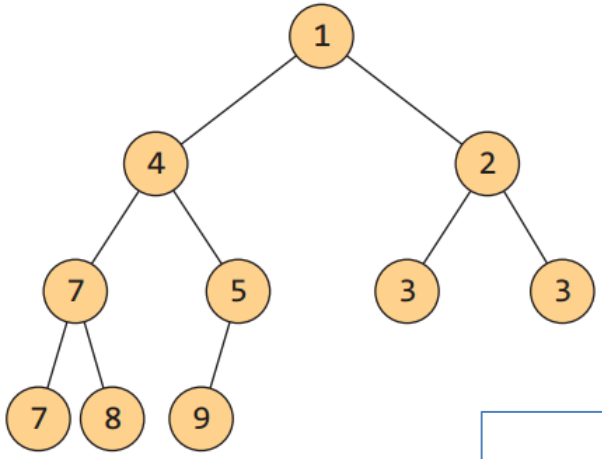
힙의 예

완전이진트리

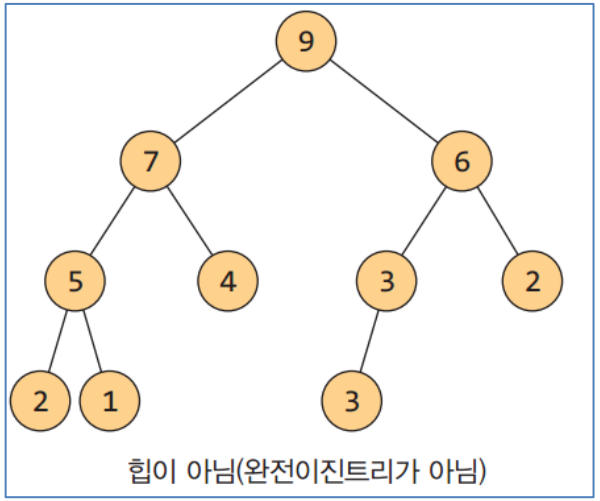
- 높이가 h일 때 레벨 1부터 h-1까지는 노드가 모두 채워짐
- 마지막 레벨 h에서는 노드가 순서대로 채워짐



최대 힙(Max Heap)



최소 힙(Min Heap)

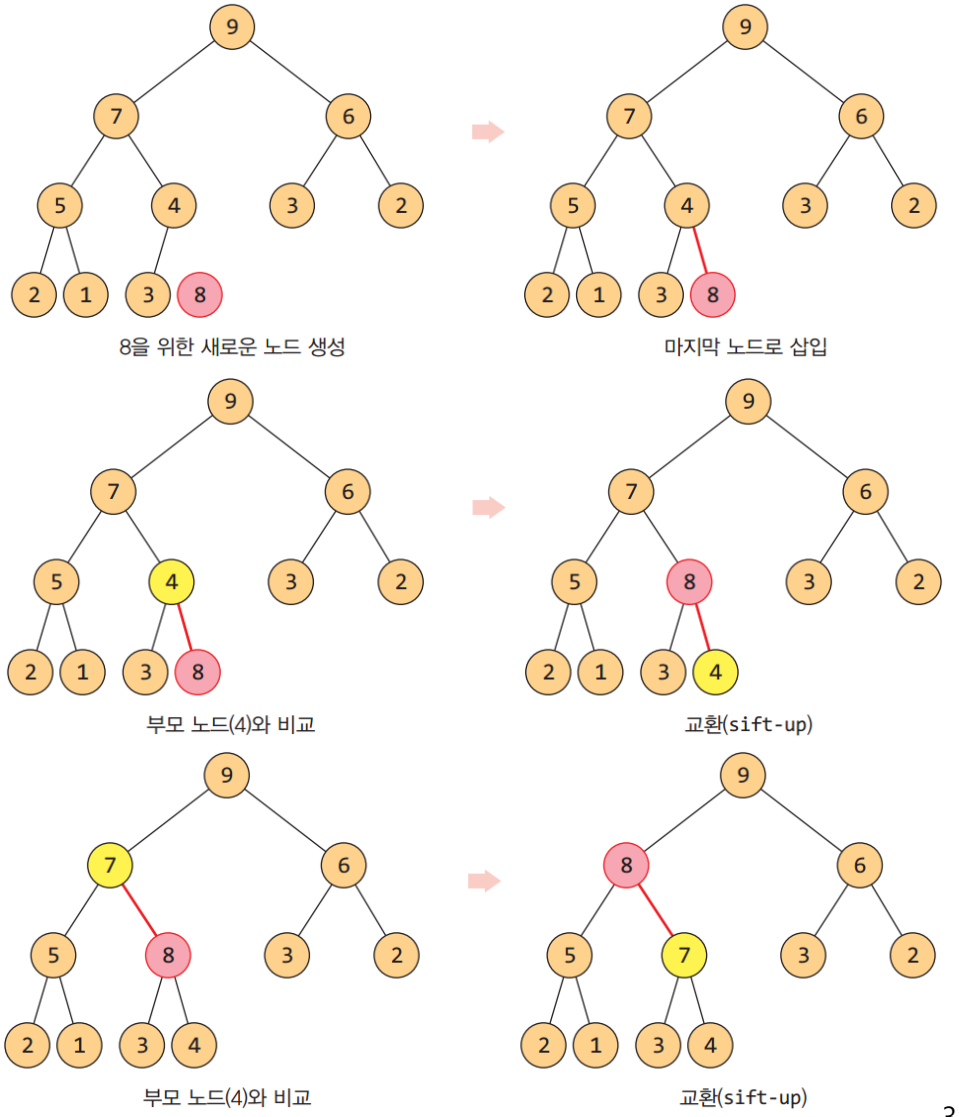


힙이 아님(완전이진트리가 아님)

힙의 연산: 삽입 연산

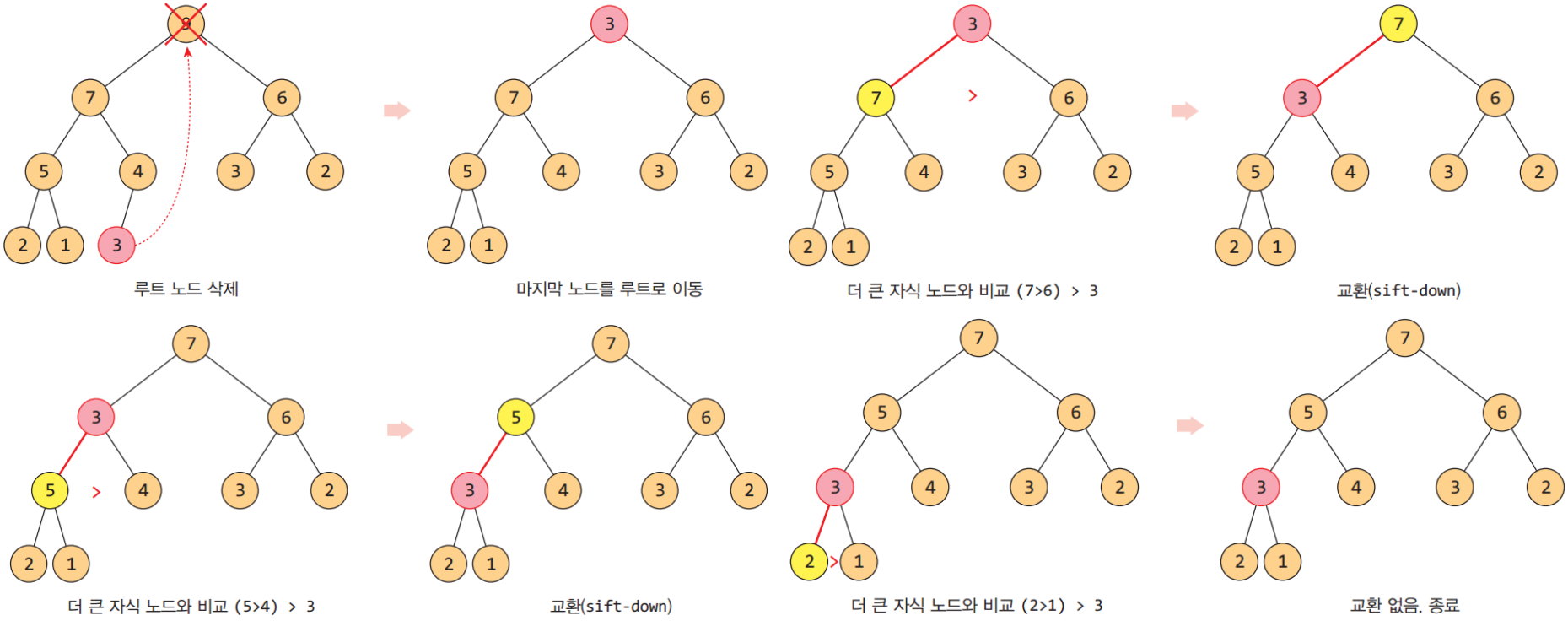
- Up-heap
  - 회사에서 신입 사원이 들어오면 일단 말단 위치에 앉힘
  - 신입 사원의 능력을 봐서 위로 승진시킴
  - (부모노드와 계속 비교)

  - 시간 복잡도: $O(logn)$



힙의 연산: 삭제 연산

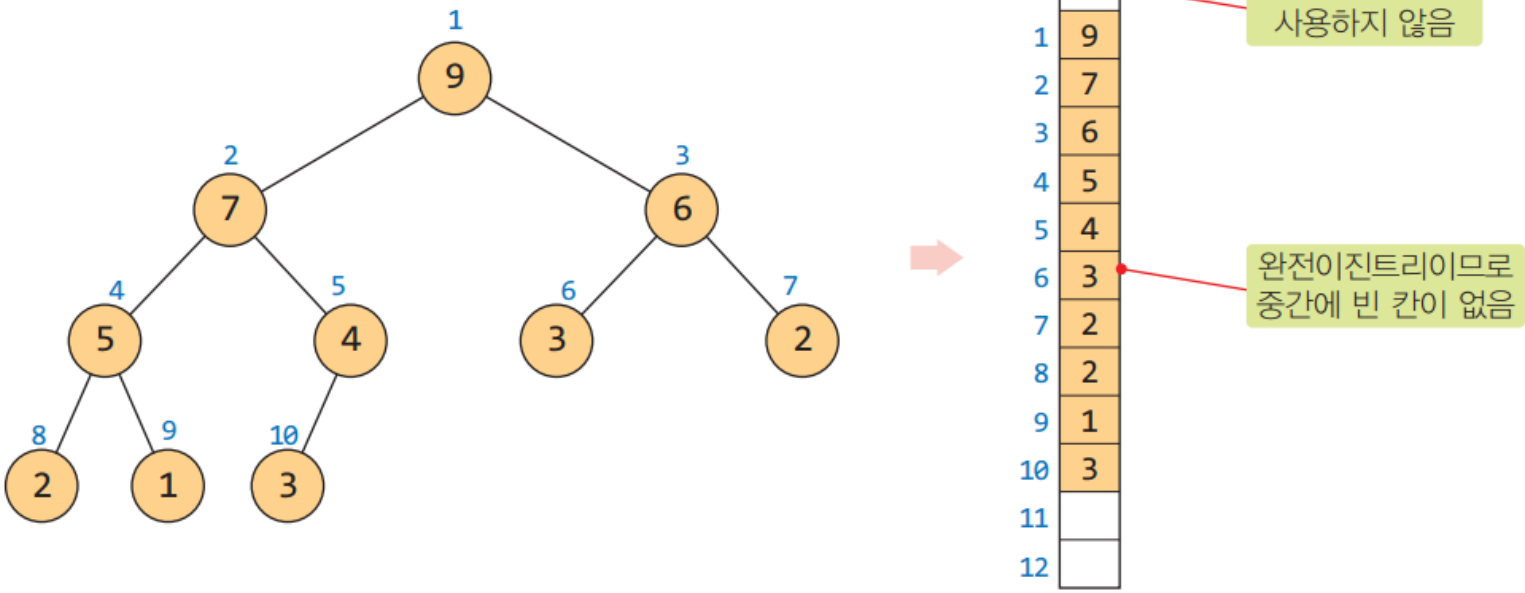
- Down-heap
  - 회사에서 신입 사원이 들어오면 사장 자리에 일단 앉힘
  - 그 후 단계적으로 강등 (더 큰 자식 노드와 계속 비교)
  - 시간 복잡도: $O(logn)$



배열을 이용한 힙의 표현 (최대 힙)

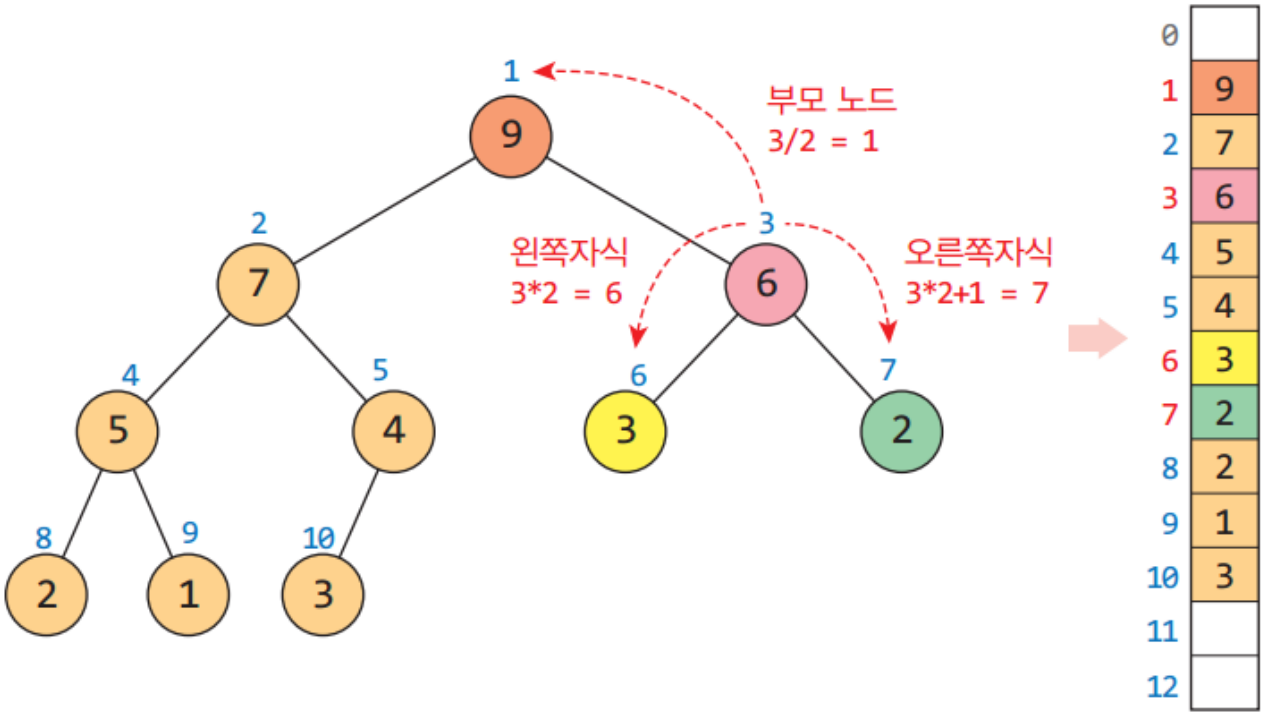


- 힙은 보통 배열을 이용하여 구현
  - 완전이진트리 ➔ 각 노드에 번호를 붙임 ➔ 배열의 인덱스
  - 트리에 노드가 추가되더라도 기존 노드의 번호는 변하지 않음



배열을 이용한 힙의 표현 (최대 힙)

- 부모/자식 노드와 인덱스의 관계
  - 왼쪽 자식의 인덱스 = (부모의 인덱스)*2
  - 오른쪽 자식의 인덱스 = (부모의 인덱스)*2 + 1
  - 부모의 인덱스 = (자식의 인덱스)/2 (나머지 제외 몫 만)



최대 힙 연산의 구현



삽입

```python
def heappush(heap, n) :
    heap.append(n)          # 맨 마지막 노드로 일단 삽입
    i = len(heap)-1         # 노드 n의 위치
    while i != 1 :          # n이 루트가 아니면 up-heap 진행
        pi = i//2           # 부모 노드의 위치
        if n <= heap[pi]:   # 부모보다 작으면 up-heap 종료
            break
        heap[i] = heap[pi]  # 부모를 끌어내림
        i = pi              # i가 부모의 인덱스가 됨
    heap[i] = n             # 마지막 위치에 n 삽입
```

삭제

```python
def heappop(heap) :
    size = len(heap) - 1    # 노드의 개수
    if size == 0 :          # 공백상태
        return None

    root = heap[1]          # 삭제할 루트 노드(사장)
    last = heap[size]       # 마지막 노드(말단사원)
    pi = 1                  # 부모 노드의 인덱스
    i = 2                   # 자식 노드의 인덱스

    while (i <= size):      # 마지막 노드 이전까지
        if i<size and heap[i] < heap[i+1]:  # right가 더 크면 i를 1 증가 (기본은 왼쪽 노드)
            i += 1          # 비교할 자식은 오른쪽 자식
        if last >= heap[i]: # 자식이 더 작으면 down-heap 종료
            break
        heap[pi] = heap[i]  # 아니면 down-heap 계속
        pi = i
        i *= 2

    heap[pi] = last         # 맨 마지막 노드를 parent위치에 복사
    heap.pop()              # 맨 마지막 노드 삭제
    return root             # 저장해두었던 루트를 반환
```

왼쪽 자식의 인덱스
= (부모의 인덱스)*2

오른쪽 자식의 인덱스
= (부모의 인덱스)*2 + 1

부모의 인덱스
= (자식의 인덱스)/2

인덱스 0은 사용하지 않아도
항목이 들어가 있어야 함
(main 에서 처리)

테스트 프로그램



아래 코드 이어서 복붙

```
if __name__ == "__main__":

    data = [2, 5, 4, 8, 9, 3, 7, 3]     # 힙에 삽입할 데이터
    heap = [0]
    print("입력: ", data)
    for e in data :                 # 모든 데이터를 힙에 삽입
        heappush(heap, e)
        print("heap: ", heap[1:])

    print("삭제: ", heappop(heap))
    print("heap: ", heap[1:])
    print("삭제: ", heappop(heap))
    print("heap: ", heap[1:])
```

```
입력:  [2, 5, 4, 8, 9, 3, 7, 3]
heap:  [2]
heap:  [5, 2]
heap:  [5, 2, 4]
heap:  [8, 5, 4, 2]
heap:  [9, 8, 4, 2, 5]
heap:  [9, 8, 4, 2, 5, 3]
heap:  [9, 8, 7, 2, 5, 3, 4]
heap:  [9, 8, 7, 3, 5, 3, 4, 2]
삭제:  9
heap:  [8, 5, 7, 3, 2, 3, 4]
삭제:  8
heap:  [7, 5, 4, 3, 2, 3]
```

우선순위 큐의 성능 비교



- 우선순위 큐의 주요 연산
  - enqueue(): 새로운 요소를 삽입하는 연산
  - dequeue(): 우선순위가 가장 높은 요소를 꺼내서 반환
  - peek(): 우선순위가 가장 높은 요소를 참조(반환)

- 다양한 방법으로 구현한 우선순위 큐의 성능 비교

표현 방법	enqueue()	dequeue()	peek()
정렬 안 된 배열	$O(1)$	$O(n)$	$O(n)$
정렬된 배열	$O(n)$	$O(1)$	$O(1)$
힙	$O(\log_2 n)$	$O(\log_2 n)$	$O(1)$

Summary



이진 트리

1. 특징
- 계층 구조는 갖는 자료 표현에 적합한 구조
- 루트와 왼쪽/오른쪽 서브 트리(또한 이진 트리)로 구성
- 검색, 우선순위 큐 활용, 압축, 수식 처리 등에 용이함

2. 연산 (순회)
- 전위 순회 : 루트 -> 왼쪽 -> 오른쪽 (트리 복사, 노드 레벨 계산, 구조화된 문서 출력)
- 중위 순회 : 왼쪽 -> 루트 -> 오른쪽 (이진 탐색 트리 정렬)
- 후위 순회 : 왼쪽 -> 오른쪽 -> 루트 (트리 삭제, 폴더 용량 계산)
- 레벨 순회 : 루트 -> 다음 레벨 왼쪽 ... 반복 (순환x, 큐를 사용해 구현)

3. 응용 (모스 부호)
- (알파벳, 모스부호) 의 튜플로 이루어진 리스트 생성 (A~Z)
- 결정 트리 생성 (A~Z 까지 모스 부호가 . 이면 왼쪽, _ 이면 오른쪽
- 입력 받은 문자열 encoding -> decoding

힙 트리

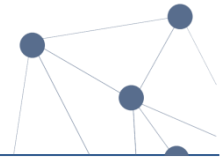
1. 특징
- 트리 중에서도 우선 순위(최대/최소)대로 데이터 찾는 데에 최적화: O(1) (배열에서 찾는건 O(n))
- 완전 이진 트리로 구성 (레벨 1 ~ 높이-1 까지는 노드 모두 채워짐 + 최상 레벨에서는 노드가 왼쪽부터 채워짐)

2. 연산
- 삽입 : 말단 ~ 맞는 자리까지 승진 (부모 노드 비교)/ 삭제 : 사장 자리 ~ 강등 (자식노드 비교)

3. 구현 (최대 힙)
- 배열의 인덱스 vs 부모/자식노드 관계 -> 코드로 구현



감사합니다!



8.6 힙의 응용: 허프만 코드

- 알파벳의 빈도를 미리 알 수 있다면?
  - 이진트리를 이용하여 이 문서를 압축하여 용량을 줄일 수 있
  - 허프만 코딩 트리



빈도수 분석

A	80
B	16
C	32
D	36
E	123
F	22
G	16
H	51
I	71
...	
Z	1

  - 예: E와 Z만 표현한다면

① 모든 문자를 7비트로 표현
7bit * (E 123회 + Z 1회) → 868비트

② E는 2비트로, Z를 20비트로 표현
2bit * (E 123회) + 20bit * (Z 1회) → 266비트

가변길이 코드



- 고정길이코드와 가변길이코드의 인코딩
  - 인코딩은 쉬움. 테이블에서 찾아 나열.

	F	A	C	E
고정길이코드:	0101	0000	0010	0100

	F	A	C	E
가변길이코드:	0111	00	0110	10

글자	빈도수	고정길이코드			가변길이코드		
		코드	비트수	전체 비트수	코드	비트수	전체 비트수
A	17	0000	4	68	00	2	34
B	3	0001	4	12	11110	5	15
C	6	0010	4	24	0110	4	24
D	9	0011	4	36	1110	4	36
E	27	0100	4	108	10	2	54
F	5	0101	4	20	0111	4	20
G	4	0110	4	16	11110	5	20
H	13	0111	4	52	010	3	39
I	15	1000	4	60	110	3	45
J	1	1001	4	4	11111	5	5
합계	100			400			292

디코딩?



- 고정길이코드의 디코딩
  - 고정된 크기(4비트)로 끊어서 읽으면 됨

	F	A	C	E
고정길이코드:	0101	0000	0010	0100

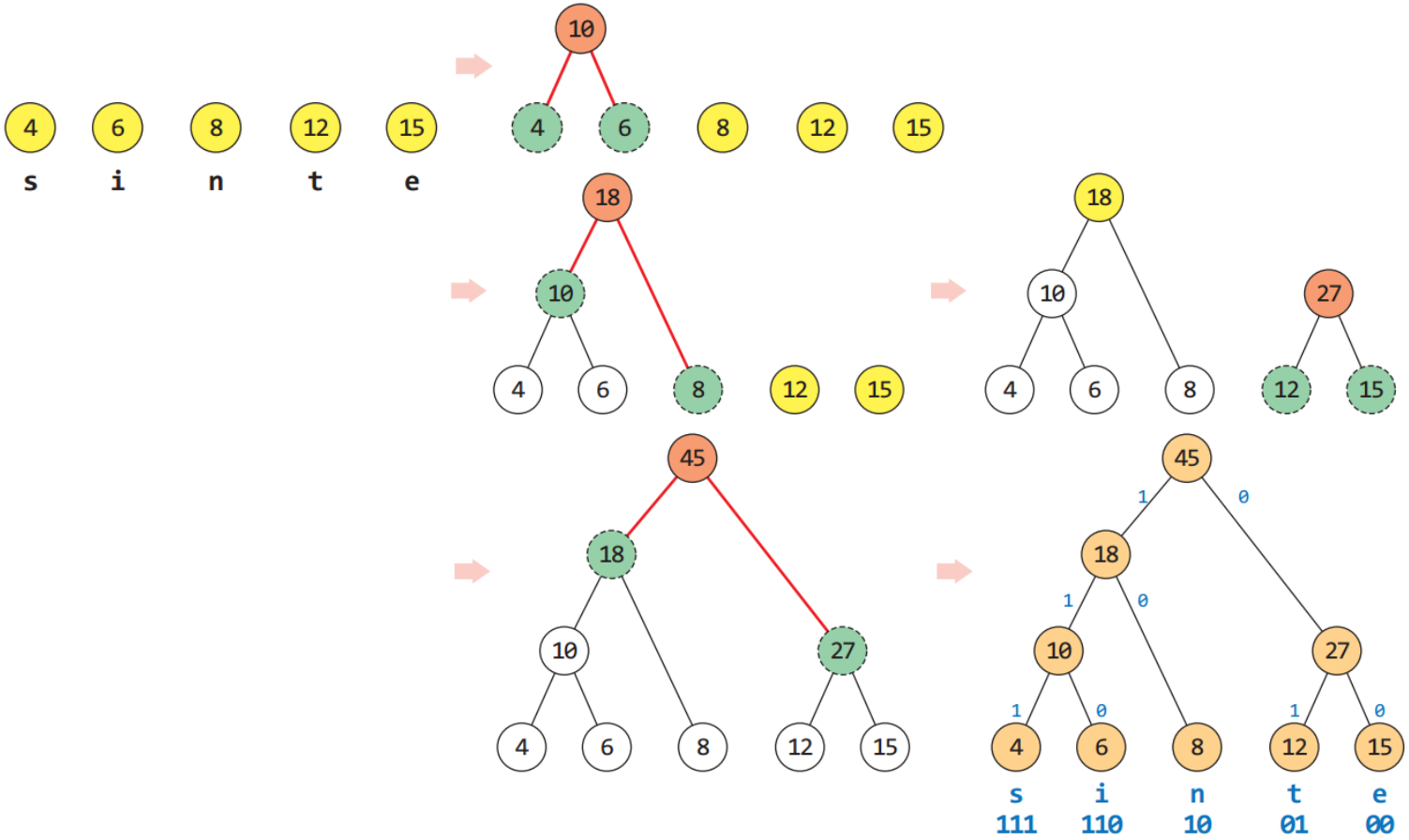
- 가변길이코드의 디코딩
  - 한 비트씩 읽으면서 테이블의 코드를 찾아야 함

	F	A	C	E
가변길이코드:	0111	00	0110	10

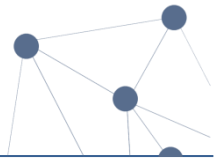
  - 모든 코드가 다른 코드의 첫 부분이 아니어야 함
  - 예: 허프만 코드

허프만 코드 생성 과정

- 최소힙에 모든 노드 삽입 → 가장 작은 두 노드를 꺼내 빈도를 더하고 최소힙에 다시 삽입 → … → 코드 부여



허프만 코딩 트리 만들기



- 최소 힙으로 구현

```
def make_tree(freq):
    heap = [0]
    for n in freq :
        heappush(heap,n)

    for i in range(1, len(freq)) :
        e1 = heappop(heap)
        e2 = heappop(heap)
        heappush(heap, e1 + e2)
        print(" (%d+%d)" % (e1, e2))

label = [ 'E', 'T', 'N', 'I', 'S' ]
freq  = [15, 12, 8, 6, 4 ]
make_tree(freq)
```

```
C:₩WINDOWS₩system32₩cmd.exe
(4+6)
(8+10)
(12+15)
(18+27)
```